Robotik Mühendisliği Serisi — Cilt 3

Donanım ve Gömülü Kontrol

Motorlar, dişliler, servolar, sensörler, mikrodenetleyici, güç

Türkçe–İngilizce karma konu anlatımı
teori · sayısal örnekler · çalışan Python ve C++ kodları

Odak: bacaklı ve humanoid robotlar

31 Ekim 2026

İçindekiler

Bu Cilt Hakkında **1**

1 DC Motor Modeli **2**

1.1 İki denklem ve nereden geldikleri . . . 2

1.2 Kararlı hal: tork-hız doğrusu . . . 2

1.3 Zaman sabitleri . . . 3

2 Dişli Kutusu, Yansıyan Atalet ve Şeffaflık **7**

2.1 1. Yansıyan atalet: $N^2 J_m$ . . . 7

2.2 2. Geri sürülebilirlik ve şeffaflık . . . 7

2.3 3. Darbe ve çarpışma . . . 8

2.4 Dişli verimi ve sürtünme . . . 8

3 Pozisyon Servosu ve Kontrol Döngüleri **9**

3.1 Kaskad kontrol: akım, hız, konum . . . 9

3.2 PWM . . . 9

3.3 Hobi servosunun içi . . . 9

4 Sensörler **14**

4.1 Enkoder ve potansiyometre . . . 14

4.2 IMU: jiroskop ve ivmeölçer . . . 14

5 Mikrodenetleyici, Sabit Frekanslı Döngü ve Haberleşme **18**

5.1 Mikrodenetleyici ile bilgisayarın farkı . . . 18

5.2 Sabit frekanslı döngü . . . 18

5.3 Bilgisayarda test edilebilir gömülü kod . . . 18

5.4 Haberleşme . . . 21

6 Güç, Besleme ve Güvenlik **22**

6.1 Güç bütçesi . . . 22

6.2 Pil . . . 22

6.3 Güvenlik . . . 22

7 Gerçek Bacaklı Robot Aktüatörleri **23**

7.1 BLDC ve alan yönelimli kontrol (FOC) 23
7.1.1 Clarke ve Park dönüşümlerinin türetilmesi 23
7.2 Quasi-direct-drive (QDD) aktüatör 23
7.3 Ara seçenek: akıllı servolar 24
7.4 Senin yol haritan 24

8 Proje P2: Masaüstü 2-DoF Bacak **25**
8.1 Hedef 25
8.2 Malzeme (3D yazıcı olmadan) 25
8.3 Adımlar 25
8.4 Başarı ölçütleri 25

9 Sorular **27**

10 Cevaplar **30**

Bu Cilt Hakkında

Cilt 3, kontrol algoritmasının ürettiği bir sayının ("diz torku 0.6 N·m olsun") gerçek bir motora, gerçek bir sensörden gelen ölçümle nasıl ulaştığını anlatıyor: motor ve dişli fiziği, hobi servolarının içi ve sınırları, enkoder/IMU ile ölçüm, mikrodenetleyicide sabit frekanslı döngü, haberleşme, güç ve güvenlik. Hedef proje **P2**: elindeki iki MG996R ve Arduino ile masaüstü 2-DoF bacak.

Staj dönemi için: Bu cilt büyük ölçüde kâğıt-kalemle çalışılabilir. Her bölümdeki "sayılar" kutularını elle tekrar hesapla; sonunda 30 soruluk bir bölüm ve cevapları var. Aynı soruların telefonda okunacak A5 sürümü ayrıca *Kitapçık 3* olarak verildi. Kodlar (Python ve C++) bilgisayar dönemine kadar sadece *okunacak*: her satırın ne yaptığını defterine yaz.

Sayıların kaynağı

MG996R için kullandığım değerler üreticinin datasheet'inden: 4.8 V'ta 9.4 kgf·cm durma torku ve 0.17 s/60° hız; 6 V'ta 11 kgf·cm, 0.14 s/60°, durma akımı 2.5 A, çalışma akımı 0.5–0.9 A, ölü bölge 5 µs, kütle 55 g. Piyasadaki kopyalar bu değerleri tutturmayabilir. Servonun *içindeki* motor direnci, tork sabiti, dişli oranı gibi değerleri üretici vermiyor; Bölüm 1'de bunları datasheet sayılarını tutturacak şekilde **seçtim**. Bu "eşdeğer model" davranışı anlamak için iyi, ama gerçek servonun parametreleri değil. Gerçek değerleri P2'de kendin ölçeceksin.

Bölüm	Konu	Kod
1	DC motor modeli, tork-hız eğrisi, güç ve ısı	`d1_motor.py/.cpp`
2	Dişli kutusu, yansıyan atalet, şeffaflık, QDD	(Bölüm 1 çıktısı)
3	Pozisyon servosu, PWM, iç kontrol döngüsü, sınırlar	`d2_servo.py/.cpp`
4	Sensörler: enkoder, potansiyometre, IMU, tamamlayıcı filtre	`d3_sensors.py`, `d3_imu.cpp`
5	Mikrodenetleyici, sabit frekanslı döngü, haberleşme	`d4_leg_sketch.cpp`
6	Güç, besleme ve güvenlik	elle hesap
7	Gerçek bacaklı robot aktüatörleri ve yol haritası	–
8	Proje P2: masaüstü 2-DoF bacak	–
9–10	Sorular ve cevaplar	–

Bölüm 1

DC Motor Modeli

Bir robotun her eklemindeki aktüatör, sonunda bir elektrik motorudur. Kontrolcü "tork" ister; motor ise gerilim ve akımla çalışır. Bu bölüm ikisi arasındaki köprüyü kuruyor. Fırçalı DC motor için anlatıyoruz; fırçasız (BLDC) motorlar, uygun elektronikle (FOC, Bölüm 7) aynı denklemlere indirgenir.

1.1 İki denklem ve nereden geldikleri

Tork: Lorentz kuvveti. Manyetik alan B içinde, akım i taşıyan ℓ boyundaki iletkene $F = Bi\ell$ kuvveti etki eder. Rotordaki tüm sargıların bu kuvvetlerinin mil etrafındaki momentleri toplanınca tork akımla orantılı çıkar: $\tau = k_t i$. k_t (tork sabiti) motorun geometrisi ve mıknatıslarıyla belirlenir.

Zıt EMK: Faraday yasası. Aynı sargılar alan içinde dönünce, içlerinde dönme hızıyla orantılı bir gerilim indüklenir: $e = k_e \omega$. Motor aynı zamanda bir jeneratördür; bu gerilim besleme gerilimine *karşı* koyar.

Elektrik devresi (Kirchhoff gerilim yasası, sargı = direnç R + endüktans L + zıt EMK kaynağı) ve **mil** (Newton'un dönme yasası, atalet J, viskoz sürtünme b, yük torku):

$$\begin{aligned} \text{elektriksel:} \quad & V = Ri + L\frac{di}{dt} + k_e\,\omega \\ \text{mekanik:} \quad & J\dot{\omega} = k_t\,i - b\,\omega - \tau_{\text{yük}} \end{aligned}$$

$k_t = k_e$ **neden?** Güç dengesi: elektrik devresinin zıt EMK'ya verdiği güç $e\,i = k_e \omega i$, mekanik tarafta üretilen güç $\tau\omega = k_t i\omega$. Kayıpsız dönüşümde ikisi eşit $\Rightarrow k_t = k_e \equiv k$ (SI birimlerinde, N·m/A = V·s/rad). Datasheet'ler farklı birim (rpm/V, oz·in/A) kullanırsa dönüştürünce eşitlik çıkar; çıkmıyorsa bir birim hatası vardır.

En önemli sonuç: tork akımla orantılıdır. Motoru bir "tork kaynağı" gibi kullanmak istiyorsan akımı kontrol etmelisin (akım döngüsü, Bölüm 3). Gerilim verirsen tork, hıza da bağlı olur.

1.2 Kararlı hal: tork-hız doğrusu

$di/dt = 0$ ve $\dot{\omega} = 0$ (ve b küçük): elektrik denkleminden $i = (V - k\omega)/R$, torkta yerine koy:

$$\tau = \frac{k}{R}(V - k\omega) \quad \Rightarrow \quad \tau_{\text{durma}} = \frac{kV}{R}\ (\omega = 0), \qquad \omega_0 = \frac{V}{k}\ (\tau = 0).$$

Tork hızla doğrusal azalır: $\tau = \tau_{\text{durma}}(1 - \omega/\omega_0)$.

En büyük güç ortada. $P = \tau\omega = \tau_{\text{durma}}(\omega - \omega^2/\omega_0)$; $dP/d\omega = \tau_{\text{durma}}(1 - 2\omega/\omega_0) = 0 \Rightarrow \omega = \omega_0/2$, $P_{\max} = \tau_{\text{durma}}\omega_0/4$.

Verim. Elektrik girişi $Vi = V\tau/k$; mekanik çıkış $\tau\omega$. Oranları $\eta_m = k\omega/V = \omega/\omega_0 = 1 - \tau/\tau_{\text{durma}}$: *motor verimi yükle doğrusal düşer* (bu basit modelde, sürtünme kayıpları hariç). Dişli verimiyle çarpınca aşağıdaki tablonun verim sütunu çıkar (%10 yükte $0.9 \cdot 0.59 = 0.53$).

Isı. Bakır kaybı $i^2R = (\tau/k)^2R$: **torkun karesiyle** büyür. Durma anında $\omega = 0$, mekanik güç sıfır, elektrik gücünün tamamı (V^2/R) ısıdır.

Sayılar: MG996R'a uydurulmuş eşdeğer motor

Seçilen değerler: $V = 4.8$ V, $R = 2.4\ \Omega$, $L = 1$ mH, $k = 0.004$ N·m/A, $J_m = 2 \times 10^{-7}$ kg·m^2, dişli $N = 195$, verim $\eta = 0.59$. (Datasheet'teki iki sayıyı tutturmak için: $N = V/(k\,\omega_0) = 4.8/(0.004 \cdot 6.16) \approx 195$, $\eta = \tau_{durma}/(N\,kV/R) \approx 0.59$.)

- Motor milinde: durma torku 8.0 mN·m, yüksüz hız 1200 rad/s (≈11 500 rpm), durma akımı 2.0 A.
- Çıkışta: durma torku 0.920 N·m = 9.38 kgf·cm (datasheet 9.4), yüksüz hız 6.15 rad/s ⇒ 60° için 0.170 s (datasheet 0.17).
- En büyük mekanik güç 1.42 W (yarı torkta, yarı hızda). Durma anında $V^2/R = 9.6$ W'ın tamamı ısı: 55 g'lık bir kutuda bu, servoyu dakikalar içinde ısıtır.

Yüke göre (çıkış torku, durma torkunun yüzdesi olarak):

Yük	hız [rad/s]	akım [A]	verim	ısı [W]
%10	5.54	0.20	%53	0.10
%25	4.62	0.50	%44	0.60
%50	3.08	1.00	%30	2.40
%75	1.54	1.50	%15	5.40

Ders: hobi servosunu sürekli olarak durma torkunun yarısının üstünde çalıştırmak, ısı ve verim açısından kötü bir fikir. Cilt 1 Bölüm 4'teki "diz bükük duran 2 kg'lık robot" örneği (dizde ~ 0.9 N·m) servoyu neredeyse durma noktasında, sürekli 9 W ısıyla çalıştırırdı.

1.3 Zaman sabitleri

Elektriksel: motor dururken ($\omega = 0$) devre $L\,di/dt + Ri = V$: birinci dereceden; çözüm $i = \frac{V}{R}(1 - e^{-t/\tau_e})$, $\tau_e = L/R$.

Mekanik: L'yi ihmal et ($i = (V - k\omega)/R$ anlık) ve mekanik denkleme koy:

$$J\dot{\omega} = \frac{k}{R}(V - k\omega) \;\Rightarrow\; \underbrace{\frac{JR}{k^2}}_{\tau_m}\dot{\omega} + \omega = \frac{V}{k}.$$

Yine birinci dereceden: hız $\omega_0(1 - e^{-t/\tau_m})$. İlginç olan: τ_m'deki "sönüm" sürtünmeden değil, *zıt EMK'dan* geliyor (k^2/R etkin bir viskoz sönüm). Küçük motorlarda $\tau_e \ll \tau_m$; bu yüzden akım döngüsü (kHz) hız döngüsünden (yüz Hz) çok daha hızlı olabilir ve kontrolde çoğu zaman L ihmal edilir.

Sayılar: basamak yanıtı

$\tau_e = 0.42$ ms. Çıkışa Cilt 1'deki uyluğu ($J = 0.004$ kg·m^2) bağlarsak motor tarafında toplam atalet $J_m + J_{uzuv}/(N^2\eta) = 2.0 \times 10^{-7} + 1.8 \times 10^{-7}$ ve $\tau_m = 56.7$ ms. 4.8 V basamakta akım 2.04 ms'de 1.94 A tepeye çıkıyor (neredeyse durma akımı: motor başta durduğu için zıt EMK yok), hız 56.7 ms'de %63'e ulaşıyor, 0.2 s sonra 1165 rad/s. Python ve C++ aynı sayıları veriyor.

Python

```python
import numpy as np

# Örnek fırçalı DC motor + dişli. Değerler MG996R datasheet'ini (4.8 V: 9.4 kgf·cm, 0.17 s/60°) tutturacak
# şekilde SEÇİLDİ; servonun gerçek iç parametreleri değildir (üretici vermiyor).
V, R, L = 4.8, 2.4, 1e-3          # gerilim [V], sargı direnci [Ω], endüktans [H]
k = 0.004                         # tork sabiti = zıt EMK sabiti [N·m/A] = [V·s/rad] (SI'da eşit)
Jm, bm = 2e-7, 0.0                # rotor ataleti [kg·m²], viskoz sürtünme
N, eta = 195.0, 0.59              # dişli oranı, dişli verimi

# (1) Kararlı hal: tork-hız doğrusu (motor milinde ve çıkış milinde)
tau_s_m = k*V/R; w0_m = V/k                                  # durma torku, yüksüz hız (motor)
tau_s = N*eta*tau_s_m; w0 = w0_m/N                           # çıkışta
print(f"motor mili:  durma torku {tau_s_m*1e3:.2f} mN·m, yüksüz hız {w0_m:.0f} rad/s ({w0_m*60/2/np.pi:.0f} rpm), durma akımı {V/R:.2f} A")
print(f"çıkış mili:  durma torku {tau_s:.3f} N·m = {tau_s/9.81*100:.2f} kgf·cm (datasheet 9.4), yüksüz hız {w0:.3f} rad/s -> 60° için {np.pi/3/w0:.3f} s (datasheet 0.17)")
P_max = tau_s*w0/4
print(f"en büyük mekanik güç (τ = τ_s/2, ω = ω0/2'de): {P_max:.3f} W;  durmada elektrik gücü V²/R = {V*V/R:.2f} W (tamamı ısı)")
for frac in [0.1, 0.25, 0.5, 0.75]:
    tau = frac*tau_s; w = w0*(1 - frac); i = tau/(N*eta*k); Pin = V*i; Pout = tau*w
    print(f"  yük = %{frac*100:3.0f} durma torku: hız {w:.2f} rad/s, akım {i:.3f} A, verim {Pout/Pin*100:5.1f} %, ısı {i*i*R:.2f} W")

# (2) Zaman sabitleri ve basamak yanıtı (yüksüz, motor tarafı): L di/dt = V - R i - k ω ;  Jt dω/dt = k i - (yük)
Jload = 0.004                                                # çıkışa bağlı uyluk ataleti (Cilt 1 Bölüm 6) [kg·m²]
Jt = Jm + Jload/(N*N*eta)                                    # motora yansıyan toplam atalet (verim yaklaşımıyla)
print(f"\nyansıyan atalet: rotor {Jm:.1e}, yük/N²η {Jload/(N*N*eta):.2e} kg·m²  ->  çıkıştan görülen rotor ataleti N²Jm = {N*N*Jm:.4f} kg·m² (uyluğun {N*N*Jm/Jload:.1f} katı!)")
tau_e = L/R; tau_mech = Jt*R/(k*k)
print(f"elektriksel zaman sabiti L/R = {tau_e*1e3:.2f} ms, mekanik zaman sabiti J R/k² = {tau_mech*1e3:.1f} ms")
dt = 1e-5; T = 0.2; n = int(T/dt); i = 0.0; w = 0.0; I = np.zeros(n); W = np.zeros(n)
for s in range(n):                                           # yarı-örtük Euler, küçük adım (L/R çok küçük!)
    i += dt*(V - R*i - k*w)/L; w += dt*(k*i - bm*w)/Jt; I[s] = i; W[s] = w
t63 = np.argmax(W >= 0.632*w0_m)*dt
print(f"basamak yanıtı: tepe akım {I.max():.2f} A (t ={np.argmax(I)*dt*1e3:.2f} ms), 0.2 s sonra hız {W[-1]:.0f} rad/s (yüksüz {w0_m:.0f}), %63'e ulaşma {t63*1e3:.1f} ms")
```

Çıktı:

```
motor mili:  durma torku 8.00 mN·m, yüksüz hız 1200 rad/s (11459 rpm), durma akımı 2.00 A
çıkış mili:  durma torku 0.920 N·m = 9.38 kgf·cm (datasheet 9.4), yüksüz hız 6.154 rad/s -> 60° için 0.170 s (datasheet 0.17)
en büyük mekanik güç (τ = τ_s/2, ω = ω0/2'de): 1.416 W;  durmada elektrik gücü V²/R = 9.60 W (tamamı ısı)
  yük = % 10 durma torku: hız 5.54 rad/s, akım 0.200 A, verim  53.1 %, ısı 0.10 W
  yük = % 25 durma torku: hız 4.62 rad/s, akım 0.500 A, verim  44.3 %, ısı 0.60 W
  yük = % 50 durma torku: hız 3.08 rad/s, akım 1.000 A, verim  29.5 %, ısı 2.40 W
  yük = % 75 durma torku: hız 1.54 rad/s, akım 1.500 A, verim  14.8 %, ısı 5.40 W

yansıyan atalet: rotor 2.0e-07, yük/N²η 1.78e-07 kg·m²  ->  çıkıştan görülen rotor ataleti N²Jm = 0.0076 kg·m² (uyluğun 1.9 katı!)
elektriksel zaman sabiti L/R = 0.42 ms, mekanik zaman sabiti J R/k² = 56.7 ms
basamak yanıtı: tepe akım 1.94 A (t =2.04 ms), 0.2 s sonra hız 1165 rad/s (yüksüz 1200), %63'e ulaşma 56.7 ms
```

C++

```cpp
// d1_motor.cpp — DC motor + dişli modeli: kararlı hal ve basamak yanıtı (Python d1_motor.py ile aynı sayılar)
#include <cmath>
#include <cstdio>

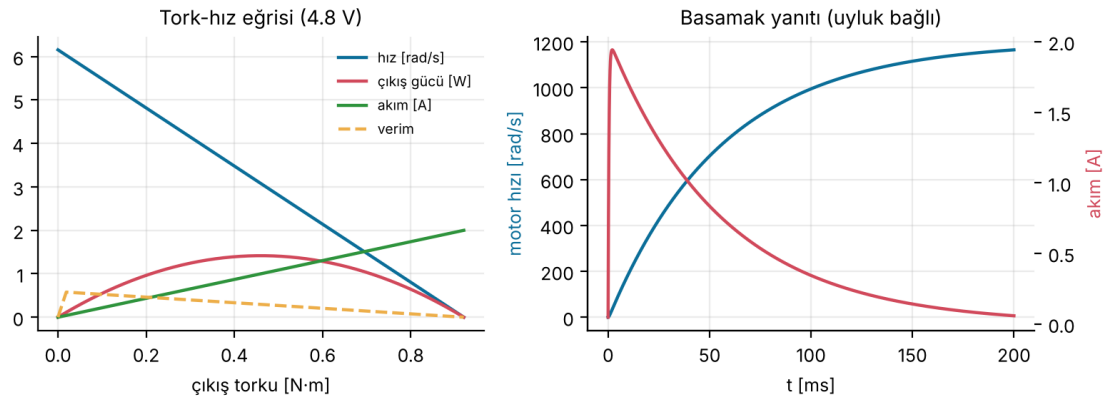
struct Motor {                          // motor + dişli parametreleri (örnek değerler, MG996R datasheet'ine uydurulmuş)
    double V = 4.8, R = 2.4, L = 1e-3, k = 0.004, Jm = 2e-7, N = 195.0, eta = 0.59;
    double stall_torque_out() const { return N * eta * k * V / R; }
    double no_load_speed_out() const { return V / k / N; }
};

int main() {
    Motor m;
    double ts = m.stall_torque_out(), w0 = m.no_load_speed_out();
    std::printf("çıkış: durma torku %.3f N·m (%.2f kgf·cm), yüksüz hız %.3f rad/s, 60° için %.3f s\n",
                ts, ts / 9.81 * 100, w0, M_PI / 3 / w0);
    std::printf("en büyük mekanik güç %.3f W, durmada ısı %.2f W\n", ts * w0 / 4, m.V * m.V / m.R);

    const double Jload = 0.004, Jt = m.Jm + Jload / (m.N * m.N * m.eta);
    double i = 0, w = 0, i_max = 0, t63 = -1;
    const double dt = 1e-5; const int n = 20000;                    // 0.2 s
    for (int s = 0; s < n; ++s) {
        i += dt * (m.V - m.R * i - m.k * w) / m.L;                   // elektriksel denklem
        w += dt * (m.k * i) / Jt;                                   // mekanik denklem
        if (i > i_max) i_max = i;
        if (t63 < 0 && w >= 0.632 * m.V / m.k) t63 = s * dt;
    }
    std::printf("basamak: tepe akım %.2f A, 0.2 s sonra hız %.0f rad/s, %%63'e ulaşma %.1f ms\n", i_max, w, t63 * 1e3);
    return 0;
}
```

Çıktı:

```
çıkış: durma torku 0.920 N·m (9.38 kgf·cm), yüksüz hız 6.154 rad/s, 60° için 0.170 s
en büyük mekanik güç 1.416 W, durmada ısı 9.60 W
basamak: tepe akım 1.94 A, 0.2 s sonra hız 1165 rad/s, %63'e ulaşma 56.7 ms
```



Şekil 1.1: Sol: çıkış milinde tork-hız doğrusu, güç (parabol, ortada en büyük), akım ve verim. Sağ: uyluk bağlıyken 4.8 V basamak yanıtı; akım hemen tepe yapıp hız arttıkça (zıt EMK) düşüyor.

Python ve C++ notu

- **Küçük zaman adımı:** $L/R = 0.42$ ms olduğundan açık integratörde $\Delta t \ll 0.4$ ms gerekir (burada 10^{-5} s). Bu "sert" (stiff) bir sistem; Cilt 1 Bölüm 6'daki enerji/kararlılık tartışmasının aynısı.
- **`struct` ile parametreler:** C++'ta `struct Motor { double V = 4.8, ...; }` varsayılan değerli üyeler ve üye fonksiyonlar (`stall_torque_out()`) tek yerde toplanır. Python'da aynısı `@dataclass` ile yapılır.
- **`%%` printf'te:** yüzde işareti yazdırmak için iki kez yazılır.

Alıştırma

(Kâğıt) (a) $V = 6$ V'ta (aynı R, k, N, η) çıkış durma torku ve yüksüz hız? Datasheet'in 6 V değerleriyle (11 kgf·cm, 0.14 s/60°) karşılaştır; model ne kadar tutuyor? **(b)** Durma torkunun %30'unda motor verimi ve ısı? **(c)** $\tau_m = JR/k^2$'yi uyluklu yük için ($J = 3.8 \cdot 10^{-7}$, motor tarafında) hesapla. **(d)** Bir motorun datasheet'i $k_v = 1000$ rpm/V diyor: k kaç N·m/A? (İpucu: $k = 1/k_v$, rad/s'ye çevir.)

Bölüm 2

Dişli Kutusu, Yansıyan Atalet ve Şeffaflık

Küçük elektrik motorları hızlı döner ama az tork üretir; eklemler ise yavaş döner ve çok tork ister. Dişli kutusu ikisini eşler: oranı N olan bir dişli için

$$\tau_{\text{çıkış}} = N\eta\,\tau_{motor}, \qquad \omega_{\text{çıkış}} = \omega_{motor}/N.$$

Bedava değil: üç gizli maliyeti var.

2.1 1. Yansıyan atalet: $N^2 J_m$

Çıkış milinden bakıldığında motor rotorunun ataleti N^2 ile büyür ($\omega_m = N\omega$, kinetik enerji $\frac{1}{2}J_m\omega_m^2 = \frac{1}{2}(N^2J_m)\omega^2$). Küçük bir rotor bile büyük oranla devasa görünür. **Çıkıştan bakınca hareket denklemi.** Motor milinde $J_m\ddot{\theta}_m = \tau_m - \tau_d/N$ (τ_d: dişlinin motora geri ilettiği yük, oranla bölünür). Çıkışta $J_\ell\ddot{\theta} = \eta\,\tau_d - \tau_{\text{yük}}$. $\theta_m = N\theta$ ile birleştirince ($\eta \approx 1$ alarak sadeleştirelim)

$$(J_\ell + N^2 J_m)\ddot{\theta} = N\tau_m - \tau_{\text{yük}}.$$

Etkin atalet $J_\ell + N^2 J_m$.

En iyi dişli oranı. Belirli bir motor torkuyla en büyük çıkış ivmesini isteyelim: $\ddot{\theta} = \frac{N\tau_m}{J_\ell + N^2 J_m}$. N'ye göre türev sıfır: $(J_\ell + N^2J_m) - 2N^2J_m = 0 \Rightarrow N^* = \sqrt{J_\ell/J_m}$: yansıyan rotor ataleti yük ataletine eşit olduğunda ("atalet eşleme"). Uyluk için $N^* = \sqrt{0.004/2\cdot 10^{-7}} = 141$; MG996R'ın 195'i bunun biraz üstünde, ama ivme farkı sadece %5. Yüksek oran seçmenin gerçek sebebi ivme değil statik tork: yerçekimini taşımak.

Sayılar: yansıyan atalet

$J_m = 2\times10^{-7}$ kg·m^2, $N = 195$: $N^2J_m = 0.0076$ kg·m^2. Cilt 1'deki uyluğun kalça etrafındaki ataleti 0.004: **rotor, bacağın kendisinden 1.9 kat daha "ağır"**. Bacağı hızlı sallamak için harcanan torkun çoğu rotoru döndürmeye gidiyor. Karşılaştırma için bir "quasi-direct-drive" (QDD) bacak aktüatörü: daha büyük rotor ama düşük oran; ör. $J_m = 5\times10^{-5}$, $N = 6$ ise $N^2J_m = 0.0018$ (örnek değerler, belirli bir ürün değil).

2.2 2. Geri sürülebilirlik ve şeffaflık

Yüksek oranlı dişliyi *çıkıştan* çevirmek zordur. Neden? Motor milindeki sabit bir sürtünme torku f_m, çıkıştan bakınca Nf_m olur; ayrıca rotoru çevirmek için çıkışta $N^2J_m\ddot{\theta}$ gerekir. $N = 195$ ve $f_m = 1$ mN·m için çıkışta 0.2 N·m'lik bir direnç: servonun durma torkunun beşte biri, sadece elle çevirebilmek için. Sonsuz (worm) dişlilerde sürtünme açısı yüzünden geri sürmek tamamen imkânsızdır. Sonuç: ayağa yerden gelen bir darbe motora "iletilmez", dişlilere biner; motor akımından dış kuvveti *hissedemezsin*. Düşük oranlı (QDD) aktüatörde ise $\tau_{\text{dış}} \approx Nk_t i$: motor akımı aynı zamanda bir kuvvet sensörüdür ("proprioceptive actuation"). MIT Cheetah ailesi ve çoğu modern dört bacaklı robot bu yüzden 6:1–10:1 civarı oranlarla çalışır.

2.3 3. Darbe ve çarpışma

Ayak yere çarptığında etkin kütle büyükse darbe kuvveti büyür. Etkin atalet $J_{uzuv} + N^2 J_m$; Cilt 1 Bölüm 7'deki KKT/temas modelinde bu terim $\mathbf{M}$'nin köşegenine eklenir (MuJoCo'da `armature` parametresi tam olarak budur). Yüksek oranlı servolarla yapılan robotlar bu yüzden yavaş ve "yumuşak olmayan" yürür.

Tasarım ödünleşimi

Yüksek dişli oranı: ucuz, küçük motorla büyük tork; ama yüksek yansıyan atalet, düşük hız, geri sürülemez, kuvvet hissedemez. Düşük oran (QDD): büyük ve pahalı motor, ama şeffaf, hızlı, darbeye dayanıklı, akımla kuvvet kontrolü. Dinamik bacaklı robotlar (koşan, zıplayan) ikinciyi seçer; senin masaüstü bacağın (P2) birinciyle başlıyor ve bunun sınırlarını ölçmek projenin öğretici kısmı.

2.4 Dişli verimi ve sürtünme

$\eta < 1$: kayıplar ısıya gider. Gerçekte kayıp sabit bir oran değil; Coulomb (yönü hıza bağlı sabit) + viskoz (hızla orantılı) sürtünme modeli daha iyidir: $\tau_f = f_c \,\mathrm{sign}(\omega) + b\,\omega$. Hobi servolarında yön değişiminde "boşluk" (backlash) da vardır: dişliler arasındaki küçük oynama, konum hatası ve titreşim kaynağıdır. P2'de bacağı elle hafifçe oynatarak boşluğu hissedebilir, telefon kamerasıyla ölçebilirsin.

Alıştırma

(Kâğıt) (a) $J_m = 5 \cdot 10^{-5}$, $J_\ell = 0.004$: en iyi oran? **(b)** $N = 6$ ve $N = 195$ için yansıyan ataletleri (Bölüm 2.1 örneğindeki J_m'lerle) uyluğun ataletiyle karşılaştır. **(c)** Akımdan kuvvet kestirimi: $N = 6$, $k = 0.1$ N·m/A, ayak bacağın 0.4 m ucunda; 1 A'lik akım değişimi ayakta kaç N'luk kuvvet değişimine karşılık gelir? **(d)** Boşluk (backlash) 1° ise 0.4 m'lik bacağın ucunda kaç mm konum belirsizliği?

Bölüm 3

Pozisyon Servosu ve Kontrol Döngüleri

3.1 Kaskad kontrol: akım, hız, konum

Endüstriyel ve robotik aktüatör sürücülerinde üç iç içe döngü vardır:

Döngü	Ölçtüğü	Ürettiği	Tipik hız
Akım (tork)	motor akımı	PWM gerilimi	10–40 kHz
Hız	enkoder türevi	akım hedefi	1–5 kHz
Konum	enkoder	hız hedefi	0.5–1 kHz

İçteki döngü dıştakinden en az 5–10 kat hızlı olmalı ki dış döngü onu "anlık" sayabilsin. Bacaklı robotlarda kontrolcü çoğu zaman en içteki *akım/tork* döngüsüne komut verir (WBC tork üretir, Cilt 6); bu yüzden tork kontrollü sürücü şarttır.

3.2 PWM

Mikrodenetleyici analog gerilim üretemez; anahtarlamayla *ortalama* gerilim üretir: periyot T'nin DT kadarında V_s, kalanında 0 ise ortalama $V_{ort} = \frac{1}{T}\int_0^T V\,dt = DV_s$. Motor endüktansı akımı süzer: anahtarlama periyodu $\tau_e = L/R$'den çok kısaysa ($T \ll 0.42$ ms, yani $f \gg 2.4$ kHz) akımdaki dalgalanma küçük kalır ve motor ortalama gerilimi "görür". Bu yüzden motor PWM'i 20 kHz civarındadır (ayrıca duyulabilir sesin üstü). İki farklı "PWM"i karıştırma:

- **Motor PWM'i:** yüksek frekans (kHz), doluluk oranı = gerilim.
- **Servo komut sinyali:** 50 Hz (20 ms periyot), *darbe genişliği* 1000–2000 µs = hedef açı. Bu bir haberleşme sinyalidir, güç taşımaz.

3.3 Hobi servosunun içi

Bir MG996R içinde: DC motor, dişli kutusu, çıkış miline bağlı potansiyometre ve küçük bir kontrol kartı. Kart her 20 ms'de gelen darbeyi okur, hedef açıyı potansiyometreyle karşılaştırır ve motoru (genelde orantısal benzeri bir kuralla) sürer. Dışarıdan bilemeyeceğin ve değiştiremeyeceğin şeyler: kazançlar, akım sınırı, iç döngü frekansı. Datasheet'teki "ölü bölge 5 µs": hedef ile ölçüm farkı bundan küçükse motor hiç sürülmez. 1000 µs = 180° ise 5 µs ≈ 0.9° (yani ±0.45°).

Model (çıkış tarafı): $V = \text{sat}(K_p e)$, $|e| <$ ölü bölge ise $V = 0$; $i = (V - kN\omega)/R$; $J_{eff}\dot{\omega} = N\eta k\, i - \tau_{\text{yük}}$, $J_{eff} = N^2 J_m + J_{uzuv}$. $K_p = 60$ V/rad bir *tahmin*.

Doymadan uzakta servo bir ikinci dereceden sistemdir. Ölü bölgeyi ve doymayı unut, yükü

sıfır al. $V = K_p e$, $i = (V - kN\omega)/R$, $J_{eff}\ddot{\theta} = N\eta k i$. Birleştir:

$$J_{eff}\ddot{\theta} = \underbrace{\frac{N\eta k}{R}K_p}_{K'_p}\, e - \underbrace{\frac{N\eta k}{R}kN}_{K'_d}\,\dot{\theta}.$$

Yani servonun içi bir PD'dir; *türev terimi motorun kendi zıt EMK'sıdır*. Sayılar: $N\eta k/R = 195 \cdot 0.59 \cdot 0.004/2.4 = 0.192$ N·m/V; $K'_p = 60 \cdot 0.192 = 11.5$ N·m/rad; $K'_d = 0.192 \cdot 0.004 \cdot 195 = 0.150$ N·m·s/rad. $J_{eff} = 0.0076 + 0.004 = 0.0116$ (Cilt 4A'nın J'si buradan). Cilt 4A Bölüm 1: $\omega_n = \sqrt{11.5/0.0116} = 31.5$ rad/s, $\zeta = 0.150/(2\sqrt{11.5 \cdot 0.0116}) = 0.20$: **az sönümlü, %50 civarı aşım** (tahmini K_p ile; gerçek servolar genelde içeride ek sönüm veya hız sınırı kullanır). Bu yüzden hobi servoları ani komutlarda "titrer"; komutu yumuşatmak (eğim sınırı) ucuz bir çaredir.

Hız sınırı. Doymada $V = \pm V_{\max}$ ve hız en fazla ω_0/N (yüksüz): 6.15 rad/s. Sinüs hedefi $A\sin(2\pi ft)$'nin tepe hızı $2\pi fA$; takip edilebilmesi için $2\pi fA < \omega_{\max}$. 30° (0.524 rad), 2 Hz: $2\pi \cdot 2 \cdot 0.524 = 6.58 > 6.15$: yetişemez.

Sayılar: servo modelinde sinüs takibi

30° genlikli sinüs hedefi, yük $\tau_{\text{yük}} = \tau_g \sin\theta$ (uzvun yerçekimi), ilk 1 s hariç:

Genlik	frekans	τ_g [N·m]	RMS hata	max hata	max akım	doyma
30°	0.5 Hz	0	1.60°	2.24°	1.40 A	%0
30°	0.5 Hz	0.6	1.70°	2.42°	1.41 A	%0
30°	1.0 Hz	0.6	3.04°	4.23°	1.65 A	%2
30°	2.0 Hz	0	**30.6°**	42.8°	**3.68 A**	%93
5°	2.0 Hz	0.6	1.23°	1.80°	0.84 A	%0

30° / 2 Hz hedefinin tepe hızı 6.58 rad/s, servonun yüksüz hızından (6.15) büyük: servo yetişemiyor, gerilim sürekli doymada, hata hedefin kendisi kadar. Max akımın durma akımını (2 A) aşması bir hata değil: motor bir yöne dönerken ters gerilim verilince zıt EMK ile besleme *toplanır* ($i = (-V - kN\omega)/R$); gerçek sürücüler bu yüzden akım sınırı koyar. Aynı frekansta 5° genlik ise rahatça izleniyor: **hız sınırı, genlik×frekans çarpımını sınırlar**. Ölü bölge: 0.3°'lik bir komut hiç hareket ettirmiyor (0.00°), 1.0°'lik komut 0.72°'de takılıyor.

Python

```python
import numpy as np

# Hobi pozisyon servosu modeli (çıkış mili tarafında). Motor/dişli değerleri d1_motor.py ile aynı (örnek).
Vs, R, k, Jm, N, eta = 4.8, 2.4, 0.004, 2e-7, 195.0, 0.59
Kp = 60.0                       # iç kontrolcü: V = Kp * hata [V/rad] (gerçek servoda bilinmez, örnek değer)
DB = np.deg2rad(0.9)            # ölü bölge: 5 µs datasheet × 0.18°/µs (1000-2000 µs <-> 0-180°)
FRAME = 0.020                   # PWM periyodu 20 ms: hedef açı sadece 50 Hz'de güncellenir
J_link = 0.004                  # bağlı uzvun ataleti [kg·m²]
Jeff = N*N*Jm + J_link          # çıkıştan görülen toplam atalet

def simulate(amp_deg, freq, tau_grav, T=4.0, dt=1e-4):
    """Sinüs hedefi takibi. tau_grav: uzvun yerçekimi torku katsayısı [N·m] (τ_yük = tau_grav·sin θ)"""
    th, w, cmd = 0.0, 0.0, 0.0; n = int(T/dt); err = []; imax = 0.0; sat = 0
    for s in range(n):
        t = s*dt
        if s % int(FRAME/dt) == 0:                    # yeni PWM darbesi geldi
```

```
            cmd = np.deg2rad(amp_deg)*np.sin(2*np.pi*freq*t)
        e = cmd - th
        V = 0.0 if abs(e) < DB/2 else np.clip(Kp*e, -Vs, Vs)
        sat += abs(V) >= Vs
        i = (V - k*N*w)/R; imax = max(imax, abs(i))       # endüktans ihmal (L/R = 0.4 ms << diğerleri)
        tau = N*eta*k*i - tau_grav*np.sin(th) - 0.002*w   # motor torku - yerçekimi - küçük sürtünme
        w += dt*tau/Jeff; th += dt*w
        if t > 1.0: err.append(np.deg2rad(amp_deg)*np.sin(2*np.pi*freq*t) - th)   # ilk 1 s geçici rejim
    err = np.array(err)
    return np.rad2deg(np.sqrt(np.mean(err**2))), np.rad2deg(np.abs(err).max()), imax, sat/n

print(f"Jeff = N²Jm + J_uzuv = {N*N*Jm:.4f} + {J_link} = {Jeff:.4f} kg·m²;  ölü bölge ±{np.rad2deg(DB)/2:.2f}°")
print(f"{'genlik':>7} {'frekans':>8} {'yük [N·m]':>10} | {'RMS hata':>9} {'max hata':>9} {'max akım':>9} {'doyma %':>8}")
for amp, f, tg in [(30, 0.5, 0.0), (30, 0.5, 0.6), (30, 1.0, 0.6), (30, 2.0, 0.0), (30, 2.0, 0.6), (5, 2.0, 0.6), (30, 0.5, 1.0)]:
    rms, mx, im, sat = simulate(amp, f, tg)
    print(f"{amp:6d}° {f:7.1f}Hz {tg:10.2f} | {rms:8.2f}° {mx:8.2f}° {im:8.2f}A {100*sat:7.1f}")
print(f"(hedefin tepe hızı: 30° @ 2 Hz -> {np.deg2rad(30)*2*np.pi*2:.2f} rad/s; servonun yüksüz hızı {Vs/(k*N):.2f} rad/s)")

# Sadece ölü bölge: 0.5° adımlık küçük komutlar
th_final = []
for step_deg in [0.3, 0.5, 1.0, 2.0]:
    th, w = 0.0, 0.0; cmd = np.deg2rad(step_deg)
    for s in range(20000):
        e = cmd - th; V = 0.0 if abs(e) < DB/2 else np.clip(Kp*e, -Vs, Vs)
        i = (V - k*N*w)/R; w += 1e-4*(N*eta*k*i - 0.002*w)/Jeff; th += 1e-4*w
    th_final.append(np.rad2deg(th))
print("küçük adım komutları [°] -> 2 s sonraki açı [°]:", [f"{a}->{b:.2f}" for a, b in zip([0.3, 0.5, 1.0, 2.0], th_final)])
```

Çıktı:

```
Jeff = N²Jm + J_uzuv = 0.0076 + 0.004 = 0.0116 kg·m²;  ölü bölge ±0.45°
 genlik  frekans  yük [N·m] |  RMS hata  max hata  max akım  doyma %
    30°     0.5Hz       0.00 |     1.60°     2.24°     1.40A     0.0
    30°     0.5Hz       0.60 |     1.70°     2.42°     1.41A     0.0
    30°     1.0Hz       0.60 |     3.04°     4.23°     1.65A     2.2
    30°     2.0Hz       0.00 |    30.64°    42.80°     3.68A    92.5
    30°     2.0Hz       0.60 |    25.18°    34.71°     3.40A    91.7
     5°     2.0Hz       0.60 |     1.23°     1.80°     0.84A     0.0
    30°     0.5Hz       1.00 |     2.06°     2.94°     1.41A     0.0
(hedefin tepe hızı: 30° @ 2 Hz -> 6.58 rad/s; servonun yüksüz hızı 6.15 rad/s)
küçük adım komutları [°] -> 2 s sonraki açı [°]: ['0.3->0.00', '0.5->0.54', '1.0->0.72', '2.0->2.33']
```

C++

```
// d2_servo.cpp — hobi servo modeli (Python d2_servo.py ile aynı): 50 Hz komut, ölü bölge, gerilim doyması
#include <algorithm>
#include <cmath>
#include <cstdio>

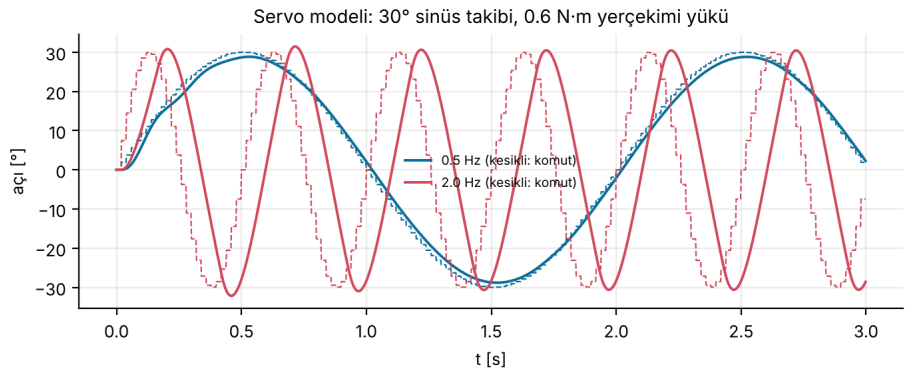
struct ServoModel {
    double Vs = 4.8, R = 2.4, k = 0.004, N = 195.0, eta = 0.59, Kp = 60.0;
    double DB = 0.9 * M_PI / 180.0, J = 195.0 * 195.0 * 2e-7 + 0.004;
    double th = 0, w = 0;
    // bir zaman adımı; i_out: o anki akım
```

```cpp
    void step(double cmd, double tau_grav, double dt, double& i_out, bool& saturated) {
        double e = cmd - th;
        double V = (std::abs(e) < DB / 2) ? 0.0 : std::clamp(Kp * e, -Vs, Vs);
        saturated = std::abs(V) >= Vs;
        i_out = (V - k * N * w) / R;
        double tau = N * eta * k * i_out - tau_grav * std::sin(th) - 0.002 * w;
        w += dt * tau / J; th += dt * w;
    }
};

int main() {
    struct Case { double amp_deg, f, tg; };
    std::printf("genlik frekans  yük | RMS hata  max hata  max akım  doyma%%\n");
    for (Case c : {Case{30, 0.5, 0.0}, Case{30, 0.5, 0.6}, Case{30, 1.0, 0.6}, Case{30, 2.0, 0.0},
Case{30, 2.0, 0.6}, Case{5, 2.0, 0.6}}) {
        ServoModel s; const double dt = 1e-4; const int n = 40000, frame = 200;
        double cmd = 0, se2 = 0, emax = 0, imax = 0; int cnt = 0, nsat = 0;
        const double A = c.amp_deg * M_PI / 180;
        for (int k = 0; k < n; ++k) {
            double t = k * dt;
            if (k % frame == 0) cmd = A * std::sin(2 * M_PI * c.f * t);   // 20 ms'de bir yeni komut
            double i; bool sat; s.step(cmd, c.tg, dt, i, sat);
            imax = std::max(imax, std::abs(i)); nsat += sat;
            if (t > 1.0) { double e = A * std::sin(2 * M_PI * c.f * t) - s.th; se2 += e * e; emax =
std::max(emax, std::abs(e)); ++cnt; }
        }
        std::printf("%5.0f° %5.1fHz %5.2f | %7.2f° %8.2f° %8.2fA %6.1f\n", c.amp_deg, c.f, c.tg,
                    std::sqrt(se2 / cnt) * 180 / M_PI, emax * 180 / M_PI, imax, 100.0 * nsat / n);
    }
    return 0;
}
```

Çıktı:

```
genlik frekans  yük | RMS hata  max hata  max akım  doyma%
   30°   0.5Hz  0.00 |    1.60°     2.24°     1.40A    0.0
   30°   0.5Hz  0.60 |    1.70°     2.42°     1.41A    0.0
   30°   1.0Hz  0.60 |    3.04°     4.23°     1.65A    2.2
   30°   2.0Hz  0.00 |   30.64°    42.80°     3.68A   92.5
   30°   2.0Hz  0.60 |   25.18°    34.71°     3.40A   91.7
    5°   2.0Hz  0.60 |    1.23°     1.80°     0.84A    0.0
```



Şekil 3.1: Servo modelinde 0.5 Hz (takip ediyor, küçük gecikme) ve 2 Hz (hız sınırına takılıyor: genlik korunuyor ama yanıt hedefin neredeyse çeyrek periyot gerisinde, bu yüzden anlık hata hedefin kendisi kadar büyük) sinüs takibi. Kesikli çizgiler 50 Hz'de güncellenen komut.

Bu bir model

Kazanç, ölü bölge davranışı ve akım sınırı tahmin. Gerçek MG996R'ın davranışını P2'de ölçeceksin: telefonun yavaş çekim kamerasıyla (ör. 240 fps) bir basamak komutuna yanıtı çek, kareleri sayarak açı-zaman grafiği çıkar ve bu modeli ona uydur (Cilt 4'teki sistem tanılama). Model ile ölçüm arasındaki fark, CV'ne yazabileceğin gerçek bir mühendislik sonucudur.

Bölüm 4

Sensörler

4.1 Enkoder ve potansiyometre

Enkoder açıyı tam sayı "sayım" olarak verir: N_{CPR} sayım/tur ise çözünürlük $q = 2\pi/N_{CPR}$. Manyetik enkoderler (12–14 bit) ve optik enkoderler (binlerce satır, 4× kodlama) yaygın. **Potansiyometre** (servoların içindeki) analog gerilim verir; ADC ile dijitale çevrilir: 10-bit ADC'de 1024 seviye.

Hız ölçmenin tuzağı: hız genelde ölçülmez, açının farkından hesaplanır: $\hat{\omega} = \Delta\theta/\Delta t$. Bir sayımlık değişim $q/\Delta t$ hızına karşılık gelir; hızlı döngüde (küçük Δt) bu sayı büyür. Çareler: daha uzun pencere, alçak geçiren filtre (gecikme ekler), sayımlar arası süreyi ölçmek, veya Kalman filtresi (Underactuated rehberi Bölüm 16–17).

Nicemleme gürültüsü. Gerçek açı bir sayım aralığının içinde düzgün dağılmış kabul edilirse, ölçülen değerle farkı $[-q/2, q/2]$'de düzgün dağılır. Düzgün dağılımın varyansı $\frac{1}{q}\int_{-q/2}^{q/2} x^2 dx = q^2/12$: **RMS nicemleme hatası** $q/\sqrt{12}$. 12 bit için $q = 1.53\cdot10^{-3}$ rad, RMS $4.4\cdot10^{-4}$ rad (0.025°): konum için mükemmel. Ama türev için kötü: ardışık iki örneğin farkı ya 0 ya $\pm q$ olur, hızın çözünürlüğü $q/\Delta t$.

Sayılar: 12-bit enkoderden hız

4096 sayım/tur: $q = 5.27$ yay-dakikası (0.088°). 1 kHz döngüde bir sayım $q/\Delta t = 1.534$ rad/s'ye karşılık geliyor; ölçmek istediğimiz hız ise ± 0.5 rad/s (yavaş bir bacak hareketi). Ham fark ya 0 ya ± 1.53 veriyor: RMS hata 0.61 rad/s (sinyalin kendisinden büyük). 10 ms pencereli fark: 0.060; $\tau = 10$ ms alçak geçiren: 0.046 rad/s, ama ~ 10 ms gecikme pahasına (1 Hz'de 3.6° faz). **Servo potansiyometresi** (10-bit, 0–180°): 1 LSB = 0.176°; tek LSB'lik gürültü 1 ms'de türevde 3 rad/s hata demek. Cilt 1'deki hesaplarda "hız ölçülüyor" dediğimiz her yer, pratikte böyle bir kestirimdir.

4.2 IMU: jiroskop ve ivmeölçer

Jiroskop açısal hızı ölçer; integre edilince yönelim verir ama *bias* (sabit küçük kayma) integralde doğrusal büyüyen hataya dönüşür. **İvmeölçer** yerçekimi yönünü ölçer (robot ivmelenmiyorsa): eğim açısını kaymasız verir ama her adım darbesinde, titreşimde gürültülüdür. İkisi birbirini tamamlar:

$$\hat{\theta}_k = \alpha(\hat{\theta}_{k-1} + \omega_{gyro}\Delta t) + (1-\alpha)\,\theta_{acc}, \qquad \tau = \frac{\alpha\Delta t}{1-\alpha}.$$

Türetme. Sürekli zamanda kestirim, jiroskopla ilerlesin ve ivmeölçere doğru yavaşça çekilsin: $\dot{\hat{\theta}} = \omega_{\text{gyro}} + \frac{1}{\tau}(\theta_{\text{acc}} - \hat{\theta})$. Laplace'ta

$$\hat{\Theta} = \underbrace{\frac{\tau s}{\tau s + 1}}_{\text{yüksek geçiren}} \frac{\Omega_{\text{gyro}}}{s} + \underbrace{\frac{1}{\tau s + 1}}_{\text{alçak geçiren}} \Theta_{\text{acc}}.$$

İki filtre toplamı 1: iki sensör de doğru açıyı ölçüyorsa ($\Omega/s = \Theta_{\text{acc}} = \Theta$) kestirim Θ, bozulmasız. Jiroskobun integrali yüksek frekansta, ivmeölçer alçak frekansta kullanılıyor. Geri Euler ile ayrıklaştırınca yukarıdaki formül ve $\alpha = \tau/(\tau + \Delta t)$, yani $\tau = \alpha\Delta t/(1-\alpha)$ çıkar. **Bias hatası:** jiroskop $\omega + b$ ölçerse hata $e = \hat{\theta} - \theta$ için $\dot{e} = b - e/\tau$; dengede $e_\infty = b\tau$.

Tamamlayıcı (complementary) filtre: τ'dan hızlı değişimlerde jiroskopa, yavaş değişimlerde ivmeölçere güvenir. Sabit kazançlı bir Kalman filtresidir. Kalıcı bias hatası $\approx b\,\tau$. (Yaw ivmeölçerle düzeltilemez: yerçekimi yaw'u görmez; manyetometre veya görsel odometri gerekir.)

Sayılar: 60 saniyelik eğim kestirimi

Gerçek eğim $\pm 10^\circ$, 0.2 Hz; jiroskop bias'ı 0.01 rad/s (0.57°/s), ivmeölçer gürültüsü 3°. 100 Hz.

- Sadece jiroskop: 60 s sonra 34.5° hata ($= 0.01 \cdot 60$ rad).
- Sadece ivmeölçer: RMS 3.0°.
- Tamamlayıcı ($\alpha = 0.98$, $\tau = 0.49$ s): RMS 0.45°; son 10 s ortalama hata 0.35° (teori $b\tau = 0.28^\circ$). C++'ta (farklı gürültü örneğiyle) RMS 0.44°.

Python

```python
import numpy as np
rng = np.random.default_rng(0)

# (1) Enkoder ile hız: 12-bit (4096 sayım/tur) manyetik enkoder, 1 kHz döngü
CPR, dt = 4096, 1e-3; q = 2*np.pi/CPR
t = np.arange(0, 2.0, dt); w_true = 0.5*np.sin(2*np.pi*1.0*t)                # gerçek hız: ±0.5 rad/s, 1 Hz
th_true = np.cumsum(w_true)*dt; counts = np.floor(th_true/q)                  # enkoder tam sayı sayım verir
w_raw = np.diff(counts, prepend=counts[0])*q/dt                               # 1 ms'lik fark
w_10 = np.zeros_like(t); w_10[10:] = (counts[10:] - counts[:-10])*q/(10*dt)   # 10 ms pencereli fark
alpha = dt/(0.01 + dt); w_lp = np.zeros_like(t)                               # 1. derece alçak geçiren (τ = 10 ms)
for k in range(1, len(t)): w_lp[k] = w_lp[k-1] + alpha*(w_raw[k] - w_lp[k-1])
rms = lambda e: np.sqrt(np.mean(e[100:]**2))
print(f"enkoder çözünürlüğü {np.rad2deg(q)*60:.2f} yay-dakika; 1 ms'de hız çözünürlüğü q/dt = {q/dt:.3f} rad/s (ölçülen hızın 3 katı!)")
print(f"hız RMS hatası: ham fark {rms(w_raw - w_true):.3f}, 10 ms pencere {rms(w_10 - w_true):.3f}, alçak geçiren τ =10 ms {rms(w_lp - w_true):.3f} rad/s")
print(f"bedel: 1. derece filtre düşük frekanslarda yaklaşık τ = 10 ms gecikme ekler (1 Hz'de faz gecikmesi {np.rad2deg(np.arctan(2*np.pi*1.0*0.01)):.1f}°)")

# (2) Potansiyometre + 10-bit ADC (servo içindeki geri bildirim)
lsb = 180/1023; print(f"\n10-bit ADC, 0-180°: 1 LSB = {lsb:.3f}°; ±1 LSB gürültüyle 1 ms'de türev gürültüsü ~ {np.deg2rad(lsb)/1e-3:.2f} rad/s")

# (3) IMU: jiroskop (bias + gürültü) ve ivmeölçer (gürültü + titreşim) ile eğim açısı, tamamlayıcı filtre
dt2, T = 0.01, 60.0; t2 = np.arange(0, T, dt2)
ang = np.deg2rad(10)*np.sin(2*np.pi*0.2*t2)                                   # gerçek gövde eğimi (pitch)
rate = np.gradient(ang, dt2)
gyro = rate + 0.01 + 0.005*rng.standard_normal(len(t2))                       # 0.01 rad/s sabit bias
acc_ang = ang + np.deg2rad(3)*rng.standard_normal(len(t2))                    # ivmeölçerden eğim: gürültülü (adım darbeleri)
g_only = np.cumsum(gyro)*dt2
a = 0.98; cf = np.zeros_like(t2)
```

```
for k in range(1, len(t2)): cf[k] = a*(cf[k-1] + gyro[k]*dt2) + (1 - a)*acc_ang[k]
d = lambda x: np.rad2deg(x)
print(f"\nIMU, 60 s: sadece jiroskop integrali son hata {d(g_only[-1] - ang[-1]):.2f}° (bias × 60 s = {d(0.01*60):.1f}°)")
print(f"  sadece ivmeölçer RMS hata {d(rms(acc_ang - ang)):.2f}°;  tamamlayıcı filtre (α =0.98, τ ={a*dt2/(1-a):.2f} s) RMS hata {d(rms(cf - ang)):.2f}°, son 10 s ortalama hata {d(np.mean((cf - ang)[-1000:])):.2f}°")
print(f"  bias'ın filtrede bıraktığı kalıcı hata (teori): bias·τ = {d(0.01*a*dt2/(1 - a)):.2f}°")
```

Çıktı:

```
enkoder çözünürlüğü 5.27 yay-dakika; 1 ms'de hız çözünürlüğü q/dt = 1.534 rad/s (ölçülen hızın 3 katı!)
hız RMS hatası: ham fark 0.610, 10 ms pencere 0.060, alçak geçiren τ =10 ms 0.046 rad/s
bedel: 1. derece filtre düşük frekanslarda yaklaşık τ = 10 ms gecikme ekler (1 Hz'de faz gecikmesi 3.6°)

10-bit ADC, 0-180°: 1 LSB = 0.176°; ±1 LSB gürültüyle 1 ms'de türev gürültüsü ~ 3.07 rad/s

IMU, 60 s: sadece jiroskop integrali son hata 34.48° (bias × 60 s = 34.4°)
  sadece ivmeölçer RMS hata 3.01°;  tamamlayıcı filtre (α =0.98, τ =0.49 s) RMS hata 0.45°, son 10 s ortalama hata 0.35°
  bias'ın filtrede bıraktığı kalıcı hata (teori): bias·τ = 0.28°
```

C++: mikrodenetleyiciye taşınabilir filtre sınıfı

```cpp
// d3_imu.cpp — tamamlayıcı (complementary) filtre: mikrodenetleyicide nasıl yazılır
#include <cmath>
#include <cstdio>
#include <random>

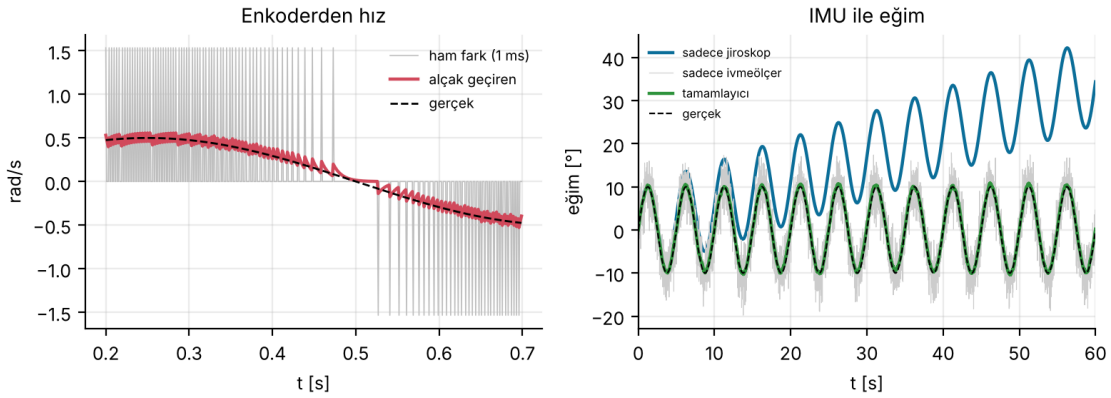
class ComplementaryFilter {
public:
    explicit ComplementaryFilter(double alpha) : alpha_(alpha) {}
    // gyro_rate [rad/s], acc_angle [rad], dt [s]  ->  güncel eğim kestirimi [rad]
    double update(double gyro_rate, double acc_angle, double dt) {
        angle_ = alpha_ * (angle_ + gyro_rate * dt) + (1.0 - alpha_) * acc_angle;
        return angle_;
    }
private:
    double alpha_;
    double angle_ = 0.0;
};

int main() {
    std::mt19937 gen(0); std::normal_distribution<double> n01(0.0, 1.0);
    const double dt = 0.01, T = 60.0, bias = 0.01, deg = M_PI / 180;
    ComplementaryFilter cf(0.98);
    double gyro_int = 0, se_cf = 0, se_acc = 0; int cnt = 0;
    double ang = 0, ang_prev = 0;
    for (int k = 1; k < (int)(T / dt); ++k) {
        double t = k * dt;
        ang = 10 * deg * std::sin(2 * M_PI * 0.2 * t);
       double rate = (ang - ang_prev) / dt; ang_prev = ang;            // gerçek açısal hız (fark ile)
        double gyro = rate + bias + 0.005 * n01(gen);
        double acc = ang + 3 * deg * n01(gen);
        gyro_int += gyro * dt;
        double est = cf.update(gyro, acc, dt);
       if (k > 100) { se_cf += (est - ang) * (est - ang); se_acc += (acc - ang) * (acc - ang); ++cnt; }
    }
    std::printf("60 s: jiroskop integrali hatası %.2f°, ivmeölçer RMS %.2f°, tamamlayıcı filtre RMS %.2f°\n",
```

```
                (gyro_int - ang) / deg, std::sqrt(se_acc / cnt) / deg, std::sqrt(se_cf / cnt) / deg);
    return 0;
}
```

Çıktı:

```
60 s: jiroskop integrali hatası 34.42°, ivmeölçer RMS 3.00°, tamamlayıcı filtre RMS 0.44°
```



Şekil 4.1: Sol: 12-bit enkoderden 1 kHz'de hız; ham fark (gri) sayım adımlarından ibaret, filtrelenmiş (turuncu) gerçeği izliyor. Sağ: jiroskop integrali kayıyor, ivmeölçer gürültülü, tamamlayıcı filtre ikisinin iyi yanını birleştiriyor.

C++ notu: sınıf ve durum

`class ComplementaryFilter` filtrenin durumunu (`angle_`) gizli (`private`) tutar, dışarıya sadece `update()` verir. Arduino'da aynı sınıfı aynen kullanabilirsin (`double` yerine `float`: AVR'de `double` zaten 4 bayttır). `explicit` kurucu, `ComplementaryFilter f = 0.98;` gibi kazara dönüşümleri engeller. Sonundaki alt çizgi (`angle_`) üye değişkeni yerel değişkenden ayırmak için yaygın bir isimlendirme.

Bölüm 5

Mikrodenetleyici, Sabit Frekanslı Döngü ve Haberleşme

5.1 Mikrodenetleyici ile bilgisayarın farkı

Arduino Uno (ATmega328P): 16 MHz, 2 KB RAM, 32 KB flash, donanımda kayan nokta yok (her `float` işlemi yazılımla, onlarca saat çevrimi). İşletim sistemi yok: kod doğrudan donanımda, tek döngüde çalışır. Avantajı *öngörülebilir zamanlama*; bir PC'deki Linux ise "ortalamada hızlı" ama ara sıra milisaniyelerce gecikebilir. Gerçek robotlarda tipik mimari: hızlı ve zamanlaması kritik iş (akım/servo döngüsü, sensör okuma) mikrodenetleyicide; yavaş ve hesap yoğun iş (planlama, MPC, kestirim) bilgisayarda; aralarında seri haberleşme.

5.2 Sabit frekanslı döngü

`delay(10)` ile yazılan döngünün periyodu 10 ms + işin süresi olur ve iş süresi değiştikçe kayar. Doğrusu: bir sonraki tetik zamanını tut ve *ona* göre çalış:

```
if ((long)(micros() - next_us) < 0) return;   // zamanı gelmedi
next_us + = PERIOD_US;                          // sabit ızgara: kayma birikmez
```

`(long)(a - b) < 0` karşılaştırması, `micros()` sayacının yaklaşık 70 dakikada bir taşmasına (overflow) dayanıklıdır: işaretsiz çıkarma, taşmadan sonra da doğru farkı verir (C'deki işaretsiz aritmetik kuralı).

5.3 Bilgisayarda test edilebilir gömülü kod

Aşağıdaki taslak, P2'nin yazılımı: 100 Hz döngü, ayak yörüngesi (yerde geri itme + havada elips), Cilt 1'deki analitik IK, eklem limitleri, açıdan darbe genişliğine çevirme. Gerçek Arduino'da çalıştıramadım; bunun yerine `millis/micros/Servo/Serial`'i taklit eden küçük bir başlık dosyası (`arduino_mock.hpp`) yazıp *aynı mantığı* bilgisayarda çalıştırdım. Bu yöntem ("donanımı taklit ederek test") gerçek projelerde de kullanılır: donanıma yüklemeden önce hataların çoğu bilgisayarda yakalanır.

Sayılar: taslağın bilgisayardaki çıktısı

Masaüstü bacak (uyluk = baldır = 0.10 m), ayak yüksekliği −0.16 m, adım 4 cm, yükseklik 2 cm, 1 s periyot. 3 s'lik sahte saatte 310 servo yazımı (yapay gecikmeler toplam süreyi ∼3.1 s'ye uzattığı için 300 değil), en büyük gecikme 410 µs (döngü periyodunun %4'ü). İstenen en büyük eklem hızı 2.10 rad/s: MG996R'ın yüksüz hızının (~ 6.2) üçte biri; Bölüm 3'teki tabloya göre takip hatası birkaç derece olmalı. Örnek komutlar: $t = 0.52$ s'de kalça 1255 µs, diz 1090 µs.

Taklit (mock) başlık dosyası

```cpp
// arduino_mock.hpp — Arduino fonksiyonlarının masaüstü (bilgisayar) taklitleri. Gerçek kartta bu dosya KULLANILMAZ:
// Arduino IDE kendi millis(), micros(), Servo ve Serial'ini sağlar. Amaç: aynı .ino mantığını bilgisayarda test etmek.
#pragma once
#include <cmath>
#include <cstdio>

namespace mock { inline unsigned long now_us = 0; }          // sahte saat (simülasyonda ilerletilir)
inline unsigned long micros() { return mock::now_us; }
inline unsigned long millis() { return mock::now_us / 1000; }
inline float constrain(float x, float lo, float hi) { return x < lo ? lo : (x > hi ? hi : x); }

class Servo {                                                // Arduino Servo kütüphanesinin taklidi
public:
    void attach(int pin) { pin_ = pin; }
    void writeMicroseconds(int us) { last_us = us; ++n_writes; }
    int pin_ = -1, last_us = 1500, n_writes = 0;
};

struct SerialMock {
    void begin(long baud) { std::printf("[Serial %ld baud]\n", baud); }
    template <typename... A> void printf(const char* f, A... a) { std::printf(f, a...); }
};
inline SerialMock Serial;
```

Arduino taslağı (bilgisayarda test sürümü)

```cpp
// d4_leg_sketch.cpp — 2 servolu masaüstü bacak (P2) için Arduino taslağı.
// Gerçek kartta: #include "arduino_mock.hpp" satırını ve en alttaki main()'i SİL, #include <Servo.h> ekle.
#include "arduino_mock.hpp"

// ---------- Bacak geometrisi ve kalibrasyon (kendi ölçümlerinle değiştir) ----------
const float L1 = 0.10f, L2 = 0.10f;           // uyluk ve baldır [m] (masaüstü bacak)
const float US_PER_RAD = 1000.0f / 3.14159f;   // 1000 µs <-> 180° (datasheet; Cilt 1 Bölüm 1'deki gibi kalibre et)
const int   US_CENTER[2] = {1500, 1500};       // eklem açısı 0 iken darbe genişliği (kalibrasyonla bulunur)
const int   SIGN[2]      = {+1, -1};           // servonun montaj yönüne göre işaret
const float Q_MIN[2] = {-1.2f, 0.10f}, Q_MAX[2] = {1.2f, 2.40f};   // eklem limitleri [rad]
const unsigned long PERIOD_US = 10000;         // 100 Hz kontrol döngüsü

Servo hip, knee;
unsigned long next_us = 0, max_late_us = 0;
float q_prev[2] = {0, 0}, max_rate = 0;

// Düzlemsel 2-uzuv IK (Cilt 1 konvansiyonu: x = L1 sin q1 + L2 sin(q1+q2), z = -L1 cos q1 - L2 cos(q1+q2))
bool ik(float x, float z, float q[2]) {
    float D = (x * x + z * z - L1 * L1 - L2 * L2) / (2 * L1 * L2);
    if (D < -1.0f || D > 1.0f) return false;                  // ulaşılamaz: komutu değiştirme
    q[1] = acosf(D);                                          // diz öne bükük (q2 > 0)
    q[0] = atan2f(x, -z) - atan2f(L2 * sinf(q[1]), L1 + L2 * cosf(q[1]));
    return true;
}

```

```
// Ayak yörüngesi: 4 cm genişlik, 2 cm adım yüksekliği, 1 s periyot (yarım elips + düz itme)
void foot_traj(float t, float& x, float& z) {
    const float z0 = -0.16f, A = 0.02f, H = 0.02f, T = 1.0f;
    float s = fmodf(t, T) / T;
    if (s < 0.5f) { float p = s / 0.5f; x = -A + 2 * A * p; z = z0; }                    // yerde: geriye it
    else { float p = (s - 0.5f) / 0.5f; x = A * cosf(3.14159f * p); z = z0 + H * sinf(3.14159f * p); }  // havada: öne taşı
    x = -x;                                                                              // ileri yürüme yönü
}

int rad_to_us(int j, float q) { return (int)(US_CENTER[j] + SIGN[j] * q * US_PER_RAD + 0.5f); }

void setup() {
    Serial.begin(115200);
    hip.attach(9); knee.attach(10);
    next_us = micros();
}

void loop() {
    unsigned long now = micros();
    if ((long)(now - next_us) < 0) return;                   // daha zamanı gelmedi (taşmaya dayanıklı karşılaştırma)
    if (now - next_us > max_late_us) max_late_us = now - next_us;
    next_us += PERIOD_US;                                    // sabit frekans: bir sonraki tetik zamanı
    float t = now * 1e-6f, x, z, q[2];
    foot_traj(t, x, z);
    if (!ik(x, z, q)) return;
    for (int j = 0; j < 2; ++j) q[j] = constrain(q[j], Q_MIN[j], Q_MAX[j]);
    for (int j = 0; j < 2; ++j) {
        float rate = fabsf(q[j] - q_prev[j]) / (PERIOD_US * 1e-6f);
        if (t > 0.05f && rate > max_rate) max_rate = rate;
        q_prev[j] = q[j];
    }
    hip.writeMicroseconds(rad_to_us(0, q[0]));
    knee.writeMicroseconds(rad_to_us(1, q[1]));
}

// ---------- Sadece bilgisayarda: sahte saatle 3 s çalıştır ----------
int main() {
    setup();
    for (int k = 0; k < 300000; ++k) {                       // 10 µs adımlarla 3 s
        mock::now_us += 10;
        if (k % 1237 == 0) mock::now_us += 400;              // ara sıra gecikme (başka iş, kesme) taklidi
        loop();
        if (k == 50000 || k == 120000 || k == 170000)
            Serial.printf("t =%.2f s  kalça =%d µs  diz =%d µs\n", mock::now_us * 1e-6, hip.last_us, knee.last_us);
    }
    Serial.printf("3 s'de servo yazma sayısı: %d (beklenen ~300), en büyük gecikme %lu µs\n", hip.n_writes, max_late_us);
    Serial.printf("en büyük istenen eklem hızı %.2f rad/s (MG996R yüksüz ~6.2 rad/s)\n", max_rate);
    return 0;
}
```

Çıktı:

```
[Serial 115200 baud]
```

```
t =0.52 s  kalça =1255 µs  diz =1090 µs
t =1.24 s  kalça =1298 µs  diz =1090 µs
t =1.76 s  kalça =1247 µs  diz =994 µs
3 s'de servo yazma sayısı: 310 (beklenen ~300), en büyük gecikme 410 µs
en büyük istenen eklem hızı 2.10 rad/s (MG996R yüksüz ~6.2 rad/s)
```

5.4 Haberleşme

Arayüz	Kablo	Tipik hız	Robotikte nerede
UART (seri)	2 (TX, RX) + GND	115.2 kbit/s – birkaç Mbit/s	Arduino ↔ bilgisayar, Dynamixel servolar (yarı çift yönlü)
I2C	2 (SDA, SCL), adresli	100 / 400 kbit/s	IMU, manyetik enkoder (AS5600), sensör kartları
SPI	4 (MOSI, MISO, SCK, CS)	1–20 Mbit/s	hızlı IMU, enkoder, SD kart
CAN (CAN-FD)	2 (diferansiyel)	1 Mbit/s (FD: 5+)	motor sürücüleri arası (bacaklı robotlarda standart); gürültüye dayanıklı, uzun kablo
EtherCAT	Ethernet kablosu	100 Mbit/s	büyük humanoidler, endüstriyel robotlar

Bant genişliği hesabı: UART'ta her bayt 10 bit (1 başlangıç + 8 veri + 1 bitiş). 115 200 baud $\Rightarrow$ 11 520 bayt/s. 100 Hz'de mesaj başına en fazla 115 bayt. 12 eklemin her biri için 2 bayt konum + 2 bayt hız gönderirsen 48 bayt + başlık: sığar; ama 1 kHz'de sığmaz. Bu yüzden 1 kHz'lik eklem döngüleri CAN veya EtherCAT ile ve sürücülerin kendi içinde koşar.

Bölüm 6

Güç, Besleme ve Güvenlik

6.1 Güç bütçesi

Her aktüatör için hem *tipik* hem *en kötü durum* (durma) akımını topla. İki MG996R (datasheet, 6 V): çalışırken 0.5–0.9 A, durmada 2.5 A. En kötü durum: $2 \times 2.5 = 5$ A. Bir USB portu 0.5 A (USB 2.0) verir; Arduino'nun 5 V pini de bu yükü taşıyamaz. Servolar **ayrı bir güç kaynağından** beslenmeli, toprak (GND) Arduino ile **ortak** olmalı (aksi halde PWM sinyalinin referansı yok).

Gerilim çökmesi (brownout): servolar aynı anda kalkınca akım sıçrar; kaynak ve kablo direnci yüzünden gerilim düşer. Arduino aynı hattan besleniyorsa reset atar ve robot "anlamsızca" yeniden başlar. Çare: ayrı besleme, servo hattına büyük bir elektrolitik kondansatör (yüzlerce–bin µF), kalın ve kısa kablolar.

Sayılar: kablo ve kaynak

Kaynağın iç direnci + kablolar toplam 0.3 Ω olsun. 5 A çekişte gerilim düşümü $5 \cdot 0.3 = 1.5$ V: 6 V'luk hat 4.5 V'a iner (servo torku orantılı düşer; Bölüm 1: $\tau_{durma} \propto V$). Aynı hattan beslenen bir 5 V regülatör girişi bunu tolere edemeyebilir. Isı: $I^2R = 25 \cdot 0.3 = 7.5$ W kablo ve kaynakta.

6.2 Pil

LiPo hücresi nominal 3.7 V (dolu 4.2 V, tamamen boş sayılması gereken 3.0–3.3 V). 2S = 7.4 V nominal: MG996R'ın üst sınırı 7.2 V olduğundan doğrudan bağlanmaz; 6 V'luk bir BEC/regülatör gerekir. Kapasite (mAh) ve C değeri: 2200 mAh, 20C bir pil $2.2 \cdot 20 = 44$ A anlık verebilir. Çalışma süresi ≈ kapasite / ortalama akım: 2.2 Ah / 1.5 A ≈ 1.5 saat (verimsizlikler hariç). LiPo güvenliği: aşırı deşarj etme, şişen pili kullanma, şarjı gözetimsiz bırakma.

6.3 Güvenlik

- **Acil durdurma:** gücü *fiziksel olarak* kesen bir anahtar (yazılım değil). Yazılım kilitlenebilir.
- **Yazılım limitleri:** eklem açısı, hız ve (varsa) akım sınırları; komut gelmezse (zaman aşımı, "watchdog") motorları güvenli duruma al.
- **İlk çalıştırma:** bacak masaya sabit, ayak boşta; önce tek eklem, küçük açılar; sonra yük.
- **Servo boynuzu ve vidası:** sıkı olmalı; boşluk ölçümü (Bölüm 2) yanlış sonuç verir.

Bölüm 7

Gerçek Bacaklı Robot Aktüatörleri

7.1 BLDC ve alan yönelimli kontrol (FOC)

Modern bacaklı robotların çoğu **fırçasız (BLDC/PMSM)** motor kullanır: fırça yok (aşınma yok), yüksek tork yoğunluğu, iyi ısı atımı. Sürmek için üç fazlı bir sürücü ve rotor açısını bilen bir enkoder gerekir. **FOC** (field oriented control), üç faz akımını rotorla dönen bir çerçeveye çevirip (Clarke ve Park dönüşümleri) tork üreten bileşeni (i_q) doğrudan kontrol eder. Sonuç: Bölüm 1'deki "tork = $k_t i$" ilişkisini bir fırçalı motor gibi kullanabilirsin. Açık kaynak/uygun fiyatlı sürücüler (ODrive, moteus, SimpleFOC gibi projeler) bu işi yapar.

7.1.1 Clarke ve Park dönüşümlerinin türetilmesi

Üç faz sargısı 120° aralıklı: faz akımları i_a, i_b, i_c ve $i_a + i_b + i_c = 0$ (yıldız bağlantı, nötrden akım çıkmaz). Her sargı, kendi ekseni yönünde akımıyla orantılı bir manyetik alan üretir; toplam alan, birim vektörleri $\mathbf{e}_a = (1, 0)$, $\mathbf{e}_b = (\cos 120°, \sin 120°)$, $\mathbf{e}_c = (\cos 240°, \sin 240°)$ olan üç vektörün akımlarla ağırlıklı toplamıdır. Bu 2-B vektörü (ölçek $\frac{2}{3}$ ile) yazmak **Clarke dönüşümüdür**:

$$i_\alpha = \tfrac{2}{3}(i_a - \tfrac{1}{2}i_b - \tfrac{1}{2}i_c) = i_a, \qquad i_\beta = \tfrac{2}{3} \cdot \tfrac{\sqrt{3}}{2}(i_b - i_c) = \tfrac{1}{\sqrt{3}}(i_b - i_c)$$

(ilk eşitlikte $i_b + i_c = -i_a$ kullanıldı). Rotor elektriksel açısı θ_e kadar dönmüş; akım vektörünü rotorla birlikte dönen eksenlere izdüşür (**Park dönüşümü**, Cilt 1'deki 2-B dönme matrisinin tersi):

$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = \begin{bmatrix} \cos\theta_e & \sin\theta_e \\ -\sin\theta_e & \cos\theta_e \end{bmatrix} \begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix}.$$

i_d mıknatıs alanıyla aynı yönde (tork üretmez, sadece ısıtır), i_q ona dik: tork $\tau = \frac{3}{2} p \lambda_m i_q$ (p kutup çifti, λ_m mıknatıs akısı), yani $\tau = k_t i_q$. Kontrolcü $i_d \to 0$, $i_q \to \tau / k_t$ için iki PI döngüsü çalıştırır; dönen çerçevede bu akımlar *sabit*tir (sinüs değil), bu yüzden basit PI yeterli. Çıkan v_d, v_q ters Park ve ters Clarke ile faz gerilimlerine (PWM) çevrilir. Rotor açısını 1–2 derece hatayla bilmek şart; bu yüzden enkoder FOC'un parçasıdır.

7.2 Quasi-direct-drive (QDD) aktüatör

Büyük çaplı, düz ("pancake") BLDC motor + tek kademeli 6:1–10:1 planet dişli + motorun hemen arkasında enkoder + entegre sürücü. Bölüm 2'deki avantajlar: düşük yansıyan atalet, geri sürülebilirlik, akımdan kuvvet kestirimi, darbe dayanıklılığı. Dezavantaj: aynı torku sürekli vermek için motor büyük olmalı; sürekli torkta ısı sınırlayıcıdır.

7.3 Ara seçenek: akıllı servolar

Dynamixel gibi "akıllı" servolar: dişli oranı yüksek (şeffaf değil) ama konum, hız, akım *okunabilir*, kazançlar ayarlanabilir, bazı modellerde akım (tork) kontrol modu var, seri hat üzerinden zincirlenir. ROBOTIS OP3 (Cilt 0'da önerilen karşılaştırma modeli) bu tür servolarla yapılmış küçük bir humanoid. Hobi servosu ile QDD arasında makul bir orta yol.

7.4 Senin yol haritan

1. **Şimdi (bütçe yok):** 2 MG996R + Arduino ile P2. Amaç: IK, zamanlama, kalibrasyon, ölçüm ve modelleme becerisi. Robotun yürümesi değil.
2. **Simülasyon:** SB-1'i MuJoCo'da Cilt 4–6 ile yürüt. MuJoCo'da aktüatörleri "ideal tork" yerine bu cildin modelleriyle (tork-hız sınırı, `armature` ile yansıyan atalet, gecikme) sınırla: simülasyonun gerçekçiliği ve sim-to-real tartışması burada başlar.
3. **Bütçe bulunursa:** tek bir QDD veya akım kontrollü aktüatörle tek bacak test düzeneği (zıplayan bacak gibi). Okul/teknokent projeleri ve hocanla konuşacağın humanoid projesi için bu sıralama, "önce ölç, sonra büyüt" ilkesine dayanıyor.

Bölüm 8

Proje P2: Masaüstü 2-DoF Bacak

8.1 Hedef

İki MG996R ile sagital düzlemde çalışan bir bacak (kalça ve diz). Ayak ucunun önceden planlanmış bir yörüngeyi ne kadar iyi izlediğini *ölçmek* ve sonuçları modelle karşılaştırmak. Bu bir yürüyen robot değil; ama "model, kod, donanım, ölçüm, karşılaştırma" döngüsünün tamamı.

8.2 Malzeme (3D yazıcı olmadan)

- Gövde/uzuvlar: 2–3 mm kontrplak, sert karton veya hırdavattan ince alüminyum lama (L ya da düz profil); servo boynuzları uzuvlara vida ve küçük cıvatalarla. Uzuv boyu 10 cm (taslaktaki `L1 = L2 = 0.10`).
- Bacağı masaya sabitleyen bir dikme (kalça servosu dikmeye vidalı).
- 6 V, en az 3–5 A'lik ayrı güç kaynağı (veya 4×AA pil yuvası kısa testler için), 1000 µF kondansatör, ortak GND.
- Ölçüm: cetvel/milimetrik kâğıt arka plan + telefon kamerası (yavaş çekim varsa daha iyi).

8.3 Adımlar

1. **Kâğıtta (staj sırasında yapabilirsin):** montaj çizimi; uzuv boyları; eklem limitleri; IK'yı kâğıtta 3 hedef için çöz (Kitapçık 1 B5.1 gibi); ayak yörüngesini kâğıda çiz ve eklem açılarını elle hesapla; güç bütçesi (Bölüm 6).
2. **Kalibrasyon:** her servo için 5–7 farklı darbe genişliği ver, açıyı fotoğraftan ölç, en küçük kareler (Cilt 1 Bölüm 1) ile `US_CENTER`, `US_PER_RAD` ve işareti bul.
3. **Tek eklem testleri:** küçük adımlar; ölü bölgeyi ölç (0.3°, 0.5°, 1° komutlar).
4. **Basamak yanıtı:** 30° basamak, yavaş çekimle çek, açı-zaman eğrisi; Bölüm 3'teki modeli uydur.
5. **Yörünge takibi:** `d4_leg_sketch` taslağını yükle; ayak ucunun izini fotoğraftan (uzun pozlama veya kare kare) çıkar; IK'nın öngördüğü yörüngeyle karşılaştır. Frekansı 0.5, 1, 2 Hz yap; hatanın nasıl büyüdüğünü çiz.
6. **Belgele:** GitHub deposu: fotoğraf/GIF, kalibrasyon tablosu, model-ölçüm grafiği, "çalışmayanlar".

8.4 Başarı ölçütleri

- 0.5 Hz'de ayak ucu hatası < 5 mm (ölçümünün hassasiyetiyle birlikte raporla).

- Basamak yanıtında model ile ölçüm arasındaki fark sayıyla ifade edilmiş (ör. “yükselme süresi model 0.12 s, ölçüm 0.15 s”).
- Servoların sıcaklığı bir dakikalık çalışmada elle tutulabilir düzeyde (değilse yük/duruş yanlış).

Bölüm 9

Sorular

Hepsi kâğıt, kalem ve hesap makinesiyle. **(K)**: kod okuma/yazma. Cevaplar bir sonraki bölümde.

D1 — Motor

D1.1 $V = 6$ V, $R = 2\ \Omega$, $k = 0.01$ N·m/A (dişlisiz). Durma torku, durma akımı, yüksüz hız (rad/s ve rpm) ve en büyük mekanik güç?

D1.2 Bir motor datasheet'inde "hız sabiti 955 rpm/V" yazıyor. Tork sabiti kaç N·m/A olmalı? (İpucu: rad/s'ye çevir.)

D1.3 D1.1'deki motor durma torkunun %50'si ve %10'u yüküyle çalışıyor. Her biri için akım, hız, çıkış gücü, giriş gücü ve verim?

D1.4 $L = 0.5$ mH, $J = 10^{-6}$ kg·m^2 ise D1.1'deki motorun elektriksel ve mekanik zaman sabitleri? Hangisi akım döngüsünün, hangisi hız döngüsünün frekansını belirler?

D1.5 $k_t = k_e$ eşitliğini güç dengesinden iki satırda göster.

D2 — Dişli

D2.1 D1.1'deki motora $N = 100$, $\eta = 0.8$ dişli takılıyor. Çıkışta durma torku ve yüksüz hız?

D2.2 Rotor ataleti $J_m = 10^{-6}$ ise çıkıştan görülen yansıyan atalet? 0.004 kg·m^2'lik uzuvla karşılaştır.

D2.3 (zor) Yük ataleti J_L, motor ataleti J_m, motor torku τ_m sabit. Yükün açısal ivmesi $\alpha_L = \frac{N\tau_m}{N^2 J_m + J_L}$. İvmeyi en büyük yapan N'yi bul. $J_L = 0.004$, $J_m = 10^{-6}$ için sayısal değer?

D2.4 Yüksek dişli oranlı bir servoda ayağa dışarıdan gelen kuvveti motor akımından neden ölçemeyiz? İki neden yaz.

D3 — Servo ve kontrol döngüleri

D3.1 12 V beslemeyle %40 doluluk oranlı PWM: motorun gördüğü ortalama gerilim?

D3.2 1000–2000 µs $\leftrightarrow$ 0–180°. 1350 µs kaç dereceye karşılık gelir? 5 µs ölü bölge kaç derece?

D3.3 Servonun en yüksek hızı 6.15 rad/s. 20° genlikli bir sinüs hedefini izleyebileceği en yüksek frekans? Yük altında hızın yarıya düştüğünü varsayarsan?

D3.4 Bir hobi servosu yeni hedefi 50 Hz'de alıyor. Bu servoyla kuracağın dış konum döngüsünün bant genişliği hakkında ne söyleyebilirsin?

D3.5 $V = 4.8$ V, $R = 2.4\ \Omega$, $k = 0.004$; motor +1000 rad/s ile dönerken sürücü aniden −4.8 V uyguluyor. O anki akım? Neden durma akımından büyük?

D4 — Sensörler

D4.1 14-bit enkoder, 2 kHz döngü: açı çözünürlüğü ve bir sayımlık hız "kuantumu"?

D4.2 10-bit ADC, 0–5 V. 1 LSB kaç mV? 270°'lik bir potansiyometre tüm aralığı kullanıyorsa 1 LSB kaç derece?

D4.3 Ucuz bir jiroskopun bias'ı 0.5°/s. 60 s integralde hata? 200 Hz'de $\alpha = 0.99$ tamamlayıcı filtrenin zaman sabiti ve bias'tan kalan kalıcı hata?

D4.4 İvmeölçer neden yaw (yön) açısını düzeltemez?

D5 — Gömülü yazılım ve haberleşme

D5.1 UART 1 Mbaud. Saniyede kaç bayt? 1 kHz'de 40 baytlık mesaj sığar mı, hattın yüzde kaçını kullanır?

D5.2 `micros()` 32-bit işaretsiz sayaç. Kaç dakikada taşar? `now = 5`, `prev = 4294967290` ise işaretsiz `now - prev` kaç olur ve neden doğrudur?

D5.3 (K) Aşağıdaki döngünün gerçek frekansı ne olur (iş 3 ms sürüyor)?

```
void loop() {
    do_control();   // 3 ms sürüyor
    delay(10);
}
```

D5.4 (K) Kâğıda yaz: merkez darbe 1480 µs, 1000 µs = π rad, işaret -1 olan bir servo için `int rad_to_us(float q)`. $q = 0.5$ rad için sonuç?

D6 — Güç ve güvenlik

D6.1 12 eklemli bir biped, her eklemde MG996R. Tipik (0.7 A) ve en kötü durum (2.5 A) toplam akım?

D6.2 2S 2200 mAh LiPo, ortalama 3 A çekiliyor. Yaklaşık çalışma süresi?

D6.3 Kaynak + kablo direnci 0.2 Ω, çekilen akım 8 A, kaynak 6 V. Servoların gördüğü gerilim? Durma torku yüzde kaçına düşer?

D6.4 Servolar ayrı bir kaynaktan beslenirken GND'nin Arduino ile ortak olması neden şart?

D7 — Aktüatörler

D7.1 FOC'ta neden sadece i_q akımı kontrol edilir; i_d neden sıfırda tutulur? (Bir cümle.)

D7.2 D2.3'ün sonucunu QDD tasarımıyla ilişkilendir: bacaklı robotlar neden "optimum" N'den daha düşük oran seçer?

D8 — Proje

D8.1 P2 bacağı: $L_1 = L_2 = 0.10$ m, ayak $(x, z) = (0,\ -0.16)$. IK ile q_1, q_2'yi bul; taslaktaki kalibrasyonla (merkez 1500 µs, $1000/\pi$ µs/rad, işaretler kalça $+1$, diz -1) darbe genişliklerini hesapla. Taslağın çıktısındaki "diz = 1090 µs" ile karşılaştır.

Bölüm 10

Cevaplar

Sayısal cevapları Python ile ayrıca kontrol ettim.

D1

D1.1 $\tau_{durma} = kV/R = 0.03$ N·m, $I_{durma} = V/R = 3$ A, $\omega_0 = V/k = 600$ rad/s = 5730 rpm, $P_{max} = \tau_{durma}\omega_0/4 = 4.5$ W.

D1.2 955 rpm/V $= 955 \cdot 2\pi/60 = 100$ rad/s/V $\Rightarrow k_e = 0.01$ V·s/rad $\Rightarrow k_t = 0.01$ N·m/A.

D1.3 %50: $i = 1.5$ A, $\omega = 300$ rad/s, $P_{\text{çıkış}} = 4.5$ W, $P_{\text{giriş}} = 9$ W, verim %50. %10: $i = 0.3$ A, $\omega = 540$ rad/s, $P_{\text{çıkış}} = 1.62$ W, $P_{\text{giriş}} = 1.8$ W, verim %90. (İdeal motorda verim $= \omega/\omega_0$: hafif yükte yüksek.)

D1.4 $\tau_e = L/R = 0.25$ ms; $\tau_m = JR/k^2 = 0.02$ s. Elektriksel zaman sabiti akım döngüsünün (kHz), mekanik zaman sabiti hız döngüsünün ölçeğini belirler.

D1.5 Elektriksel olarak dönüşen güç = (zıt EMK) × akım $= k_e\omega\, i$; mekanik güç $= \tau\omega = k_t i\, \omega$. Kayıpsız dönüşümde eşit $\Rightarrow k_t = k_e$.

D2

D2.1 $0.03 \cdot 100 \cdot 0.8 = 2.4$ N·m; $600/100 = 6$ rad/s.

D2.2 $N^2 J_m = 10^4 \cdot 10^{-6} = 0.01$ kg·m^2: uzvun 2.5 katı.

D2.3 $\frac{d\alpha_L}{dN} = 0 \Rightarrow \tau_m(N^2 J_m + J_L) - N\tau_m(2NJ_m) = 0 \Rightarrow N^* = \sqrt{J_L/J_m}$ ("atalet eşleştirme": yansıyan rotor ataleti yük ataletine eşit). Sayısal: $\sqrt{0.004/10^{-6}} = 63.2$.

D2.4 (1) Dış tork motora $1/N$ ile küçülerek ulaşır, akımdaki değişim küçük; (2) dişli sürtünmesi ve verim kaybı (yön ve hıza bağlı) bu küçük sinyali bastırır; ayrıca yüksek oranlı dişli geri sürülmeyi zorlaştırır. Hobi servosunda üstelik akım dışarıdan okunamaz.

D3

D3.1 $0.4 \cdot 12 = 4.8$ V.

D3.2 $(1350 - 1000) \cdot 0.18 = 63°$. 5 µs $= 0.9°$.

D3.3 Tepe hız $A \cdot 2\pi f \le \omega_{max}$: $f \le 6.15/(0.349 \cdot 2\pi) = 2.80$ Hz. Yük altında yarı hızla ≈ 1.4 Hz. (Bölüm 3'teki tablo: 30°'de 2 Hz başarısız, 5°'de başarılı.)

D3.4 Komut 50 Hz'de geliyorsa dış döngü en iyi ihtimalle bunun 5–10'da biri, yani birkaç Hz bant genişliğine sahip olabilir; üstelik 20 ms'ye kadar gecikme ekler (gecikme payı, Underactuated rehberi Bölüm 15).

D3.5 $i = (-4.8 - 0.004 \cdot 1000)/2.4 = -3.67$ A. Zıt EMK (+4 V) ile ters besleme aynı yönde toplanıyor; durmada sadece 4.8 V vardı. Sürücüler bu yüzden akım sınırlar.

D4

D4.1 $2\pi/16384 = 3.83 \times 10^{-4}$ rad (0.022°); hız kuantumu $q/\Delta t = 0.767$ rad/s.

D4.2 $5/1024 = 4.88$ mV; $270/1024 = 0.264°$.

D4.3 30°. $\tau = \alpha\Delta t/(1-\alpha) = 0.99 \cdot 0.005/0.01 = 0.495$ s; kalıcı hata $\approx 0.5 \cdot 0.495 = 0.25°$.

D4.4 Yerçekimi vektörü dikey eksen etrafındaki dönmeden etkilenmez: robot yerinde dönse de ivmeölçer aynı şeyi ölçer.

D5

D5.1 Her bayt 10 bit ⇒ 100 000 bayt/s. $40 \cdot 1000 = 40\,000$ bayt/s: sığar, hattın %40'ı.

D5.2 2^{32} µs = 4295 s = 71.6 dakika. İşaretsiz aritmetik 2^{32} modunda: $5 - 4294967290 \equiv 11$: gerçekten 11 µs geçmiş (taşmadan 6 µs önce + 5 µs sonra). Bu yüzden "şimdi − önceki" her zaman doğru, "şimdi > hedef" karşılaştırması ise taşmada yanlıştır.

D5.3 Periyot $3 + 10 = 13$ ms ⇒ 76.9 Hz (100 değil); iş süresi değiştikçe frekans da değişir.

D5.4

```
int rad_to_us(float q) {
    const float US_PER_RAD = 1000.0f / 3.14159265f;
    return (int)(1480.0f - q * US_PER_RAD + 0.5f);   // +0.5: yuvarlama
}
```

$q = 0.5$: $1480 - 159.15 = 1320.8 \rightarrow 1321$ µs.

D6

D6.1 Tipik $12 \cdot 0.7 = 8.4$ A; en kötü durum $12 \cdot 2.5 = 30$ A.

D6.2 $2.2/3 = 0.73$ saat ≈ 44 dakika (deşarj sınırları ve verimsizlik nedeniyle daha az).

D6.3 $6 - 8 \cdot 0.2 = 4.4$ V; durma torku gerilimle orantılı: %73'e düşer.

D6.4 PWM sinyali bir *gerilim farkı*: Arduino'nun 5 V'u servo kartına göre ölçülür. Ortak referans yoksa servo sinyali yanlış veya hiç okur.

D7

D7.1 Tork, rotor manyetik alanına dik akım bileşeniyle (i_q) üretilir; i_d tork üretmez, sadece ısı (yüksek hızda alan zayıflatma hariç).

D7.2 Atalet eşleştirme sadece "en hızlı ivme"yi optimize eder. Bacaklı robotlar ayrıca şeffaflık (kuvvet hissetme), darbe dayanıklılığı ve geri sürülebilirlik ister; bunların hepsi düşük N'yi ödüllendirir. Bu yüzden N^*'den küçük oran + büyük motor seçilir.

D8

D8.1 $D = (0.0256 - 0.02)/0.02 = 0.28$, $q_2 = \arccos 0.28 = 1.287$ rad; $q_1 = \text{atan2}(0, 0.16) - \text{atan2}(0.1 \sin q_2,\ 0.1 + 0.1 \cos q_2) = 0 - 0.644 = -0.644$ rad. Kalça: $1500 + (-0.644) \cdot 318.3 = 1295$ µs; diz: $1500 - 1.287 \cdot 318.3 = 1090$ µs: taslağın çıktısıyla aynı (ayak $x \approx 0$ iken).